



视觉3 (Visual Odometry 3)

王永才

目录

直接法

VO前端设计



直接法 (Direct Visual Odometry)

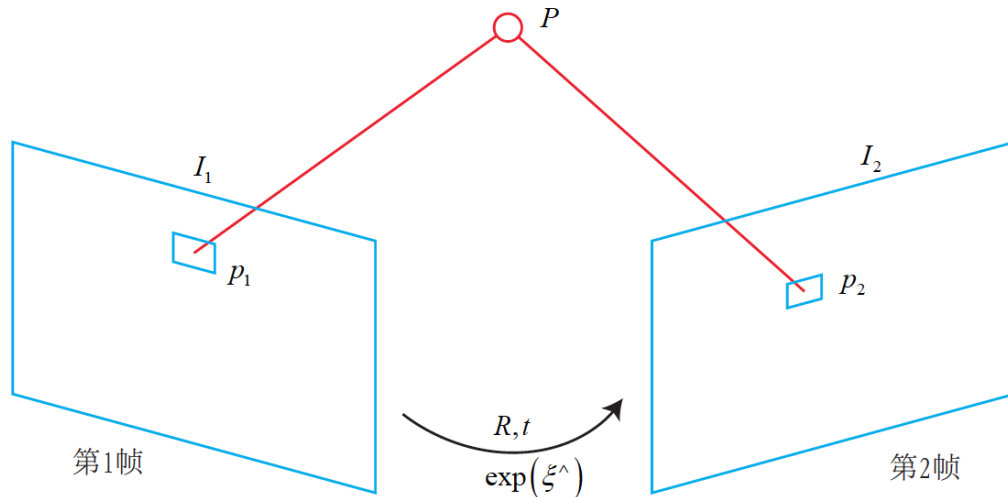
王永才



直接法

- 光流仅估计了像素间的平移，但
 - 没有用到相机本身的几何结构
 - 没有考虑到相机的旋转和图像的缩放
 - 直接法则考虑了这些信息

直接法



直接法的推导

假设有两个帧，运动未知，但有初始估计 R, t ，并且知道 P

第1帧上看到了点 P ，投影为 p_1

按照初始估计， P 在第2帧上投影为 p_2

投影关系：

$$p_1 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_1 = \frac{1}{Z_1} K P,$$
$$p_2 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_2 = \frac{1}{Z_2} K (R P + t) = \frac{1}{Z_2} K (\exp(\xi^\wedge) P)_{1:3}.$$

直接法

- 同时求由相机1到相机2的位姿变化和特征点的对应关系

$$p_1 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_1 = \frac{1}{Z_1} K P,$$

$$p_2 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_2 = \frac{1}{Z_2} K (R P + t) = \frac{1}{Z_2} K (\exp(\xi^\wedge) P)_{1:3}.$$

我们实际上并不知道对应关系

- 在相机1随机产生很多非边缘点，在相机2找对应最小光度误差点。

- 为了估计相机的运动，建立光度误差最小化问题

最小化光度误差： $e = I_1(p_1) - I_2(p_2)$ 。

- 找到对应点的同时计算出相机的位姿变换矩阵。

- 待估计的量为相机运动 ξ ，我们关心误差相对于相机运动的导数

$$\min_{\xi} J(\xi) = \sum_{i=1}^N e_i^T e_i, \quad e_i = I_1(p_{1,i}) - I_2(p_{2,i}).$$

$$\begin{aligned}
e(\xi \oplus \delta\xi) &= I_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - I_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\delta\xi^\wedge) \exp(\xi^\wedge) \mathbf{P} \right) \\
&\approx I_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - I_2 \left(\frac{1}{Z_2} \mathbf{K} (1 + \delta\xi^\wedge) \exp(\xi^\wedge) \mathbf{P} \right) \\
&= I_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - I_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\xi^\wedge) \mathbf{P} + \frac{1}{Z_2} \mathbf{K} \delta\xi^\wedge \exp(\xi^\wedge) \mathbf{P} \right).
\end{aligned}$$

类似于PnP, 记

$$\mathbf{q} = \delta\xi^\wedge \exp(\xi^\wedge) \mathbf{P},$$

$$\mathbf{u} = \frac{1}{Z_2} \mathbf{K} \mathbf{q}.$$

这里的 $\mathbf{q}$ 为 $\mathbf{P}$ 在扰动之后, 位于第二个相机坐标系下的坐标, 而 $\mathbf{u}$ 为它的像素坐标

利用一阶泰勒展开，有：

$$\begin{aligned}
 e(\boldsymbol{\xi} \oplus \delta\boldsymbol{\xi}) &= \mathbf{I}_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - \mathbf{I}_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P} + \mathbf{u} \right) \\
 &\approx \mathbf{I}_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - \mathbf{I}_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P} \right) - \frac{\partial \mathbf{I}_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \delta\boldsymbol{\xi}} \delta\boldsymbol{\xi} \\
 &= e(\boldsymbol{\xi}) - \frac{\partial \mathbf{I}_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \delta\boldsymbol{\xi}} \delta\boldsymbol{\xi}.
 \end{aligned}$$

$$\text{其中 } e(\boldsymbol{\xi}) = \mathbf{I}_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - \mathbf{I}_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P} \right)$$

我们看到，一阶导数由于链式法则分成了三项，而这三项都是容易计算的：

1. $\partial \mathbf{I}_2 / \partial \mathbf{u}$ 为 $\mathbf{u}$ 处的像素梯度；
2. $\partial \mathbf{u} / \partial \mathbf{q}$ 为投影方程关于相机坐标系下的三维点的导数。记 $\mathbf{q} = [X, Y, Z]^T$ ，根据上一节的推导，导数为：

$$\frac{\partial \mathbf{u}}{\partial \mathbf{q}} = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \end{bmatrix}$$

3. $\partial \mathbf{q} / \partial \delta \boldsymbol{\xi}$ 为变换后的三维点对变换的导数，这在李代数章节已经介绍过了：

$$\frac{\partial \mathbf{q}}{\partial \delta \boldsymbol{\xi}} = [\mathbf{I}, -\mathbf{q}^{\wedge}]$$

在实践中，由于后两项只与三维点 $\mathbf{q}$ 有关，而与图像无关，我们经常把它合并在一起：

$$\frac{\partial \mathbf{u}}{\partial \delta \boldsymbol{\xi}} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}$$

于是，我们推导了误差相对于李代数的雅可比矩阵：

$$\mathbf{J} = -\frac{\partial \mathbf{I}_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \delta \boldsymbol{\xi}}.$$

对于 N 个点的问题，我们可以用这种方法**计算优化问题的雅可比**，然后使用 G-N 或 L-M 计算增量，迭代求解。

直接法中

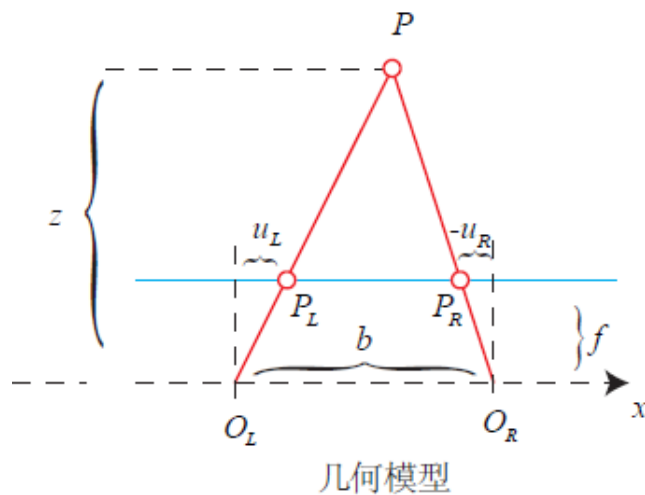
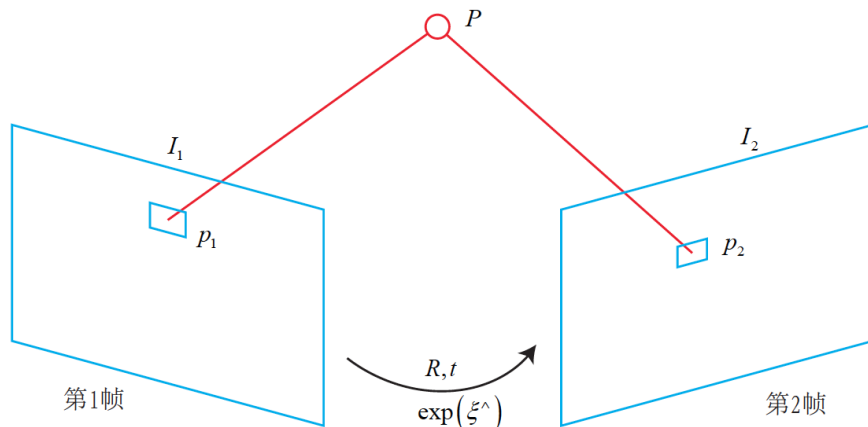
在相机1中选择的点的3D空间位置怎么来？

1. 在RGB-D中，任意像素的3D位置是可以直接获得的。

2. 在双目相机中，可以根据视差来获得像素深度。

$$z = \frac{fb}{d}, \quad d = u_L - u_R.$$

3. 单目相机中，就比较困难，还必须考虑深度带来的不确定性，第13章中详细介绍。



直接法的分类

根据 P 的来源，可以把直接法分为三类：

1. P 来自于稀疏关键点，我们称之为**稀疏直接法**。通常我们使用数百个至上千个关键点，这种稀疏直接法不必计算描述子，并且只使用数百个像素，因此速度最快，但只能计算稀疏的重构。
2. P 来自部分像素。我们看到在导数公式中，如果像素梯度为零，整一项雅可比就为零，不会对计算运动增量有任何贡献。因此，可以考虑**只使用带有梯度的像素点，舍弃像素梯度不明显的地方。这称之为半稠密 (Semi-Dense) 的直接法**，可以重构一个半稠密结构。
3. P 为所有像素，称为**稠密直接法**。稠密重构需要计算所有像素(一般几十万至几百万个)，因此多数不能在现有的 CPU 上实时计算，需要 GPU 的加速。但是，如前面所讨论的，梯度不明显的点，在运动估计中不会有太大贡献，在重构时也会难以估计位置。

直接法的实验



当前时刻左目相机图像



双目视差

1. 根据双目视差，可以首先求出当前图像中**采样点的空间位置**

$$z = \frac{fb}{d}, \quad d = u_L - u_R, \quad x = (u - c_x)z/f_x, \quad y = (v - c_y)z/f_y$$

2. 在如下图像中**通过像素差最小化，找出对应点并计算相机的位姿变换矩阵**



直接计算出相机位姿变换矩阵

$$\min_{\xi} J(\xi) = \sum_{i=1}^N e_i^T e_i, \quad e_i = \mathbf{I}_1(p_{1,i}) - \mathbf{I}_2(p_{2,i}).$$

直接法算法1

$$\min_{\xi} J(\xi) = \sum_{i=1}^N e_i^T e_i, \quad e_i = I_1(p_{1,i}) - I_2(p_{2,i}).$$

for iter=0; iter<iterations; iter ++

for i=1... n //基于所有点的测量求最优增量

$$e_i = I_1 \left(\frac{1}{Z_1} K P_i \right) - I_2 \left(\frac{1}{Z_2} K \exp(\xi^\wedge) P_i \right)$$

$$J_i = -\frac{\partial I_2}{\partial \mathbf{u}} \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}$$

$$H+ = J_i^T J_i$$

$$b+ = J_i^T e_i$$

$$\text{cost} += e_i^{T*} e_i$$

endfor

$$\Delta \xi = -H^{-1} b \quad // \text{将最优增量应用在估计的变换矩阵上}$$

$$T_{est} = \exp(\Delta \xi) T_{est} \quad // \text{如果收敛或者达到最大迭代次数则退出}$$

If cost > lastcost break;

If Converge($\Delta \xi$) break;

endfor

slambook2/ch8/direct_
method.cpp

调用G2O实现直接法

在使用 g2o 之前，需要把直接法抽象成一个图优化问题。

直接法是由以下顶点和边组成的：

- 优化变量为一个相机位姿，因此需要一个位姿顶点。由于我们在推导中使用了李代数，故程序中使用李代数表达的 $SE(3)$ 位姿顶点。使用“**VertexSE3Expmap**”作为相机位姿。
- 误差项为单个像素的光度误差。由于整个优化过程中 $I_1(p_1)$ 保持不变，我们可以把它当成一个固定的预设值，然后调整相机位姿，使 $I_2(p_2)$ 接近这个值。于是，这种边只连接一个顶点，为一元边。
- 由于 g2o 中本身没有计算光度误差的边，需要自己定义一种新的边。

直接法的图优化求解分析

直接法图优化问题是由一个相机位姿顶点与许多条一元边组成的。

如果使用稀疏的直接法，那我们大约会有几百至几千条这样的边；稠密直接法则会有几十万条边。

优化问题对应的线性方程是计算李代数增量，本身规模不大(6×6)。

主要的 计算时间会花费在每条边的误差与雅可比的计算上。

定义直接法的光度误差边

任务是：

1. 继承 BaseUnaryEdge类
2. 实现它的两个虚函数： **computeError()**, **linearizeOplus()**

//继承自g2o:BaseUnaryEdge

//类的成员包括：(1)世界坐标的一个点； (2) 相机内参矩阵； (3) 参考图像

```
class EdgeSE3ProjectDirect: public BaseUnaryEdge< 1, double, VertexSE3Expmap>
{
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW

    EdgeSE3ProjectDirect() {}

    EdgeSE3ProjectDirect ( Eigen::Vector3d point, float fx, float fy, float cx, float cy, cv::Mat*
image )
: x_world_ ( point ), fx_ ( fx ), fy_ ( fy ), cx_ ( cx ), cy_ ( cy ), image_ ( image )
{}
}
```

```

virtual void computeError()
{
    const VertexSE3Expmap* v = static_cast<const VertexSE3Expmap*> ( _vertices[0] );

    Eigen::Vector3d x_local = v->estimate().map ( x_world_ );
    //获得x_world_ 的相机坐标系坐标

    float x = x_local[0]*fx_/x_local[2] + cx_;
    float y = x_local[1]*fy_/x_local[2] + cy_;
    //获得x_world_的相平面投影坐标

    // check x,y is in the image
    if ( x-4<0 || ( x+4 ) >image_->cols || ( y-4 ) <0 || ( y+4 ) >image_->rows )
    {
        _error ( 0,0 ) = 0.0;
        this->setLevel ( 1 );
    }
    else
    {
        _error ( 0,0 ) = getPixelValue ( x,y ) - _measurement;
        //计算光度误差
    }
}
}

```

```

virtual void linearizeOplus( )
{
    if ( level() == 1
        _jacobianOplusXi = Eigen::Matrix<double, 1, 6>::Zero(); return;
    VertexSE3Expmap* vtx = static_cast<VertexSE3Expmap*> ( _vertices[0] );
    Eigen::Vector3d xyz_trans = vtx->estimate().map ( x_world_ ); // q in book

    double x = xyz_trans[0];
    double y = xyz_trans[1];
    double invz = 1.0/xyz_trans[2];
    double invz_2 = invz*invz;
    float u = x*fx_*invz + cx_;
    float v = y*fy_*invz + cy_;
    Eigen::Matrix<double, 2, 6> jacobian_uv_ksai;

    jacobian_uv_ksai ( 0,0 ) = - x*y*invz_2 *fx_;
    jacobian_uv_ksai ( 0,1 ) = ( 1+ ( x*x*invz_2 ) ) *fx_;
    jacobian_uv_ksai ( 0,2 ) = - y*invz *fx_;
    jacobian_uv_ksai ( 0,3 ) = invz *fx_;
    jacobian_uv_ksai ( 0,4 ) = 0;
    jacobian_uv_ksai ( 0,5 ) = -x*invz_2 *fx_;

    jacobian_uv_ksai ( 1,0 ) = - ( 1+y*y*invz_2 ) *fy_;
    jacobian_uv_ksai ( 1,1 ) = x*y*invz_2 *fy_;
    jacobian_uv_ksai ( 1,2 ) = x*invz *fy_;
    jacobian_uv_ksai ( 1,3 ) = 0;
    jacobian_uv_ksai ( 1,4 ) = invz *fy_;
    jacobian_uv_ksai ( 1,5 ) = -y*invz_2 *fy_;

    Eigen::Matrix<double, 1, 2> jacobian_pixel_uv;

    jacobian_pixel_uv ( 0,0 ) = ( getPixelValue ( u+1,v )-getPixelValue ( u-1,v ) ) /2;
    jacobian_pixel_uv ( 0,1 ) = ( getPixelValue ( u,v+1 )-getPixelValue ( u,v-1 ) ) /2;

    _jacobianOplusXi = jacobian_pixel_uv*jacobian_uv_ksai;
}

```

$$\begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}$$

$$\frac{\partial I_2}{\partial \mathbf{u}}$$

$$\mathbf{J} = -\frac{\partial I_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \delta \boldsymbol{\xi}}.$$

```

int main ( int argc, char** argv )
{
    srand ( ( unsigned int ) time ( 0 ) );
    string path_to_dataset = argv[1];
    string associate_file = path_to_dataset + "/associate.txt";
    ifstream fin ( associate_file );
    string rgb_file, depth_file, time_rgb, time_depth;
    cv::Mat color, depth, gray;
    vector<Measurement> measurements;
    // 相机内参
    Eigen::Matrix3f K;
    K<<fx,0.f,cx,0.f,fy,cy,0.f,0.f,1.0f;
    Eigen::Isometry3d Tcw = Eigen::Isometry3d::Identity();
    cv::Mat prev_color;
    for ( int index=0; index<10; index++ )
    {
        cout<<"***** loop "<<index<<" *****"<<endl;
        fin>>time_rgb>>rgb_file>>time_depth>>depth_file;
        color = cv::imread ( path_to_dataset+"/"+rgb_file );
        depth = cv::imread ( path_to_dataset+"/"+depth_file, -1 );
        cv::cvtColor ( color, gray, cv::COLOR_BGR2GRAY );
        if ( index ==0 )
        {
            // 对第一帧提取FAST特征点
            vector<cv::KeyPoint> keypoints;
            cv::Ptr<cv::FastFeatureDetector> detector = cv::FastFeatureDetector::create();
            detector->detect ( color, keypoints );
            for ( auto kp:keypoints )
            {
                Eigen::Vector3d p3d = project2Dto3D ( kp.pt.x, kp.pt.y, d, fx, fy, cx, cy, depth_scale );
                float grayscale = float ( gray.ptr<uchar> ( cvRound ( kp.pt.y ) ) [ cvRound ( kp.pt.x ) ] );
                measurements.push_back ( Measurement ( p3d, grayscale ) );
            }
            prev_color = color.clone();
            continue;
        }
        poseEstimationDirect ( measurements, &gray, K, Tcw );
        cout<<"Tcw="<<Tcw.matrix() <<endl;
    }
}

```

实践直接法

直接法的直观解释

- 像素灰度引导着优化的方向
- 要使优化成立，必须保证从初始估计到最优估计中间的梯度一直下降
- 这很容易受到图像非凸性的影响（可部分地由金字塔减轻）

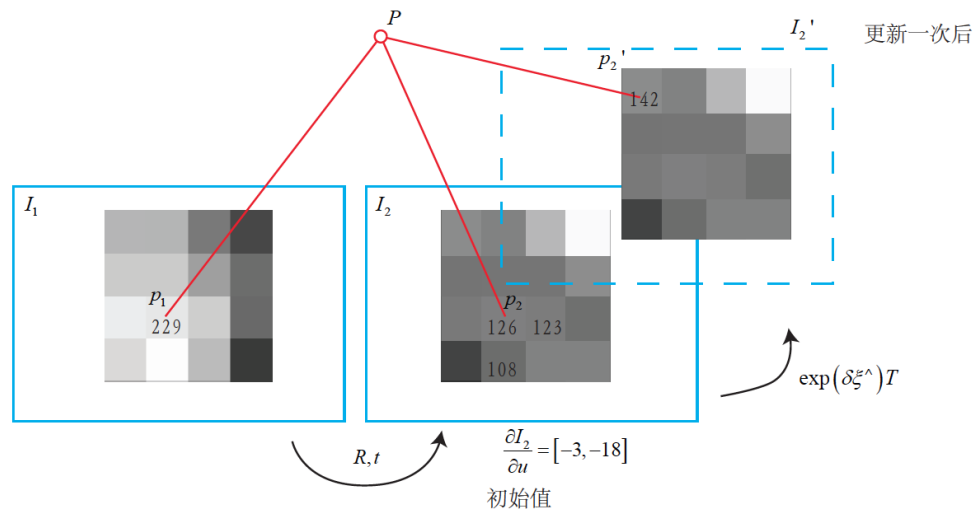
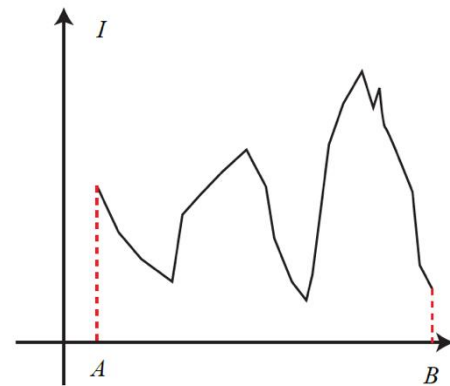
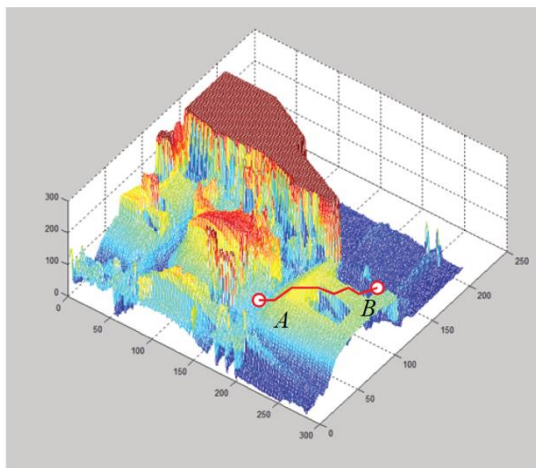


图 8-6 一次迭代的图形化显示。



直接法总结

- 可以看到，直接法的雅可比项有一个图像梯度因子
 - 因此，在图像梯度不明显的地方，对相机运动估计的贡献就小
- 根据使用的图像信息不同，可分为：
 - 稀疏直接法：只处理稀疏角点或关键点
 - 稠密直接法：使用所有像素
 - 半稠密直接法：使用部分梯度明显的像素
- 优势
 - 省略特征提取的时间
 - 只需有像素梯度而不必是角点（对白墙等地方有较好效果）
 - 可稠密或半稠密
- 劣势
 - 灰度不变难以满足（易受曝光和模糊影响）
 - 单像素区分性差
 - 图像非凸性

小结

- 2D-2D: 对极几何, 8点法, 单应矩阵方法
- 3D-2D: PnP, DLT方法, 重投影误差最小化方法
- 3D-3D: ICP, SVD方法, 对齐误差最小化方法
- 2D-2D: 用光流找对应点, 用对极几何算位姿变换
- 3D-2D: 用直接法, 直接用重投影误差最小化方法, 计算对应点和变换矩阵

目录

直接法

VO前端设计